```
!pip install -q transformers tokenizers sentencepiece spacy gensim nltk pandas
!python -m spacy download en_core_web_sm
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 40.5 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

```
from google.colab import files
uploaded = files.upload()   # select input_sub_word_data.txt
```

```
Choose files input_sub_word_data.txt
input_sub_word_data.txt(text/plain) - 6803 bytes, last modified: 12/10/2026 - 100% done
Saving input_sub_word_data.txt to input_sub_word_data (1).txt
```

```
with open("input_sub_word_data.txt", "r", encoding="utf-8") as f:
    text = f.read()
print("Input Text:\n", text)
```

subword units and how these units can be converted into token IDs for use

```
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
tokens = tok.tokenize(text)
ids = tok.convert_tokens_to_ids(tokens)

print("Tokens   :", tokens)
print("Token IDs:", ids)
print("\n{:<20}{}".format("Token", "ID"))
for t, i in zip(tokens, ids):
    print("{:<20}{}".format(t, i))
```

```
word                4775
Ġunits              4991
Ġand                290
Ġhow                703
Ġthese              777
Ġunits              4991
Ġcan                460
Ġbe                 307
Ġconverted          11513
Ġinto               656
Ġtoken              11241
ĠIDs                32373
Ġfor                329
Ġuse                779
Ċ                   198
in                  259
Ġnatural            3288
Ġlanguage           3303
Ġprocessing         7587
Ġsystems            3341
                    13
```

```
from tokenizers import Tokenizer, models, trainers, pre_tokenizers

bpe = Tokenizer(models.BPE(unk_token="[UNK]"))
bpe.pre_tokenizer = pre_tokenizers.Whitespace()
trainer = trainers.BpeTrainer(vocab_size=300, min_frequency=1,
                              special_tokens=["[UNK]", "[PAD]"])

lines = [l.strip() for l in text.split("\n") if l.strip()]
bpe.train_from_iterator(lines, trainer)

enc = bpe.encode(text)
print("Tokens   :", enc.tokens)
print("Token IDs:", enc.ids)
print("\n{:<20}{}".format("Token", "ID"))
for t, i in zip(enc.tokens, enc.ids):
    print("{:<20}{}".format(t, i))
print("\nVocabulary size:", bpe.get_vocab_size())
```

```
p                   36
ur                  90
p                   36
o                   35
se                  78
of                  86
th                  55
is                  79
experiment          294
is                  79
to                  59
understand          214
h                   28
ow                  130
BPE                 124
learn               95
s                   39
re                  61
us                  101
able                271
subword             115
units               141
and                 70
h                   28
ow                  130
th                  55
ese                 231
units               141
can                 83
be                  76
conver              276
ted                 256
into                182
token               72
ID                  296
s                   39
for                 100
use                 153
in                  48
natural             223
language            139
processing          189
sy                  225
ste                 270
ms                  250
.                   4

Vocabulary size: 300
```

```
from transformers import AutoTokenizer
```

```
sp_tok = AutoTokenizer.from_pretrained("t5-small")   # T5 uses a SentencePiece model
tokens = sp_tok.tokenize(text)
ids = sp_tok.convert_tokens_to_ids(tokens)

print("Tokens   :", tokens)
print("Token IDs:", ids)
print("\n{:<20}{}".format("Token", "ID"))
for t, i in zip(tokens, ids):
    print("{:<20}{}".format(t, i))
```

```
_units              3173
_can                54
_be                 36
_converted          12069
_into               139
_token              14145
_ID                 4699
s                   7
_for                21
_use                169
_in                 16
_natural            793
_language           1612
_processing         3026
_systems            1002
.                   5
```

```
import sentencepiece as spm

with open("sp_train.txt", "w", encoding="utf-8") as f:
    f.write(text)

spm.SentencePieceTrainer.train(
    input="sp_train.txt",
    model_prefix="sp_model",
    vocab_size=100,
    model_type="unigram",
    hard_vocab_limit=False   # needed because the corpus is tiny
)

sp = spm.SentencePieceProcessor(model_file="sp_model.model")
tokens = sp.encode(text, out_type=str)
ids = sp.encode(text, out_type=int)

print("Tokens   :", tokens)
print("Token IDs:", ids)
print("\n{:<20}{}".format("Token", "ID"))
for t, i in zip(tokens, ids):
    print("{:<20}{}".format(t, i))
```

```
h                98
o                11
w                96
_the             19
s                5
e                4
_units           43
_can             29
_be              28
_con             51
ve               27
r                6
t                9
e                4
d                13
_into            60
_token           32
_                3
I                88
D                86
s                5
_for             50
_                3
u                12
s                5
e                4
_                3
i                7
n                10
_natural         68
_language        39
_                3
p                17
r                6
o                11
c                15
e                4
s                5
s                5
ing              21
_                3
s                5
y                30
s                5
t                9
e                4
m                20
s                5
.                16
```

```
import spacy
import pandas as pd

nlp = spacy.load("en_core_web_sm")
sentence = "The young student is reading an interesting book in the library."
doc = nlp(sentence)

seen, rows = set(), []
for token in doc:
    if token.text.lower() not in seen:
        seen.add(token.text.lower())
        rows.append([token.text, token.pos_, token.tag_, spacy.explain(token.pos_), spacy.explain(token.tag_)])

df = pd.DataFrame(rows, columns=["Token", "POS", "Fine Tag", "POS Description", "Tag Description"])
print(df.to_string(index=False))
```

```
      Token   POS Fine Tag POS Description                                    Tag Description
        The   DET       DT      determiner                                         determiner
      young   ADJ       JJ       adjective adjective (English), other noun-modifier (Chinese)
    student  NOUN       NN            noun                                  noun, singular or mass
         is   AUX      VBZ       auxiliary                 verb, 3rd person singular present
    reading  VERB      VBG            verb                verb, gerund or present participle
         an   DET       DT      determiner                                         determiner
interesting   ADJ       JJ       adjective adjective (English), other noun-modifier (Chinese)
       book  NOUN       NN            noun                                  noun, singular or mass
         in   ADP       IN      adposition          conjunction, subordinating or preposition
    library  NOUN       NN            noun                                  noun, singular or mass
          . PUNCT        .     punctuation                punctuation mark, sentence closer
```

```
import nltk
import pandas as pd
from nltk.tokenize import word_tokenize

penn = {
 "CC":"Coordinating conjunction","CD":"Cardinal number","DT":"Determiner","EX":"Existential there",
 "FW":"Foreign word","IN":"Preposition / subordinating conjunction","JJ":"Adjective",
 "JJR":"Adjective, comparative","JJS":"Adjective, superlative","MD":"Modal verb",
 "NN":"Noun, singular","NNS":"Noun, plural","NNP":"Proper noun, singular","NNPS":"Proper noun, plural",
 "PDT":"Predeterminer","POS":"Possessive ending","PRP":"Personal pronoun","PRP$":"Possessive pronoun",
 "RB":"Adverb","RBR":"Adverb, comparative","RBS":"Adverb, superlative","RP":"Particle","TO":"to",
 "UH":"Interjection","VB":"Verb, base form","VBD":"Verb, past tense","VBG":"Verb, gerund / present participle",
 "VBN":"Verb, past participle","VBP":"Verb, non-3rd person singular present",
 "VBZ":"Verb, 3rd person singular present","WDT":"Wh-determiner","WP":"Wh-pronoun",
 "WP$":"Possessive wh-pronoun","WRB":"Wh-adverb",".":"Sentence-final punctuation",",":"Comma"
}
```

```
sentence = "The young student is reading an interesting book in the library."
tagged = nltk.pos_tag(word_tokenize(sentence))

seen, rows = set(), []
for word, tag in tagged:
    if word.lower() not in seen:
        seen.add(word.lower())
        rows.append([word, tag, penn.get(tag, "Other")])

df = pd.DataFrame(rows, columns=["Token", "POS Tag", "Description"])
print(df.to_string(index=False))
```

```
      Token POS Tag                           Description
        The      DT                           Determiner
      young      JJ                            Adjective
    student      NN                         Noun, singular
         is     VBZ       Verb, 3rd person singular present
    reading     VBG       Verb, gerund / present participle
         an      DT                           Determiner
interesting      JJ                            Adjective
       book      NN                         Noun, singular
         in      IN Preposition / subordinating conjunction
    library      NN                         Noun, singular
          .       .              Sentence-final punctuation
```

```
import spacy
import pandas as pd
from collections import Counter

nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

counts = Counter((t.text, t.pos_) for t in doc if not t.is_space)
rows = [[tok, pos, spacy.explain(pos), freq] for (tok, pos), freq in counts.items()]

df = pd.DataFrame(rows, columns=["Token", "POS", "Description", "Frequency"])
df = df.sort_values("Frequency", ascending=False).reset_index(drop=True)
print(df.to_string(index=False))
```

```
          occurring  VERB                 verb          1
             However   ADV               adverb          1
           separating  VERB                 verb          1
           punctuation  NOUN                 noun          1
           normalizing   ADJ            adjective          1
                spaces  NOUN                 noun          1
           unnecessary   ADJ            adjective          1
              removing  VERB                 verb          1
            converting  VERB                 verb          1
               include  VERB                 verb          1
                  Text  NOUN                 noun          1
             allowing  VERB                 verb          1
                still   ADV               adverb          1
               reduce  VERB                 verb          1
            extremely   ADV               adverb          1
            formation  NOUN                 noun          1
            languages  NOUN                 noun          1
              compare  VERB                 verb          1
           pretrained  VERB                 verb          1
              without   ADP          adposition          1
         implementing  VERB                 verb          1
                above   ADJ            adjective          1
           laboratory  NOUN                 noun          1
              changes  NOUN                 noun          1
              behaves  VERB                 verb          1
             students  NOUN                 noun          1
                 help  VERB                 verb          1
         measurements  NOUN                 noun          1
            frequency  NOUN                 noun          1
                  per   ADP          adposition          1
              average   ADJ            adjective          1
         Tokenization PROPN       proper noun          1
              amounts  NOUN                 noun          1
      representations  NOUN                 noun          1
           algorithms  NOUN                 noun          1
              Machine  NOUN                 noun          1
              systems  NOUN                 noun          1
                  use  NOUN                 noun          1
             reusable   ADJ            adjective          1
               learns  VERB                 verb          1
              purpose  NOUN                 noun          1
        modernization  NOUN                 noun          1
         hydrokinetic   ADJ            adjective          1
            renewable   ADJ            adjective          1
         unbelievable   ADJ            adjective          1
       computationally   ADV               adverb          1
        naturalization  NOUN                 noun          1
                tested  VERB                 verb          1
          vocabularies  NOUN                 noun          1
```

```
corpus_text = """Natural language processing is interesting
Natural language processing is useful
```

```
I love machine learning
Machine learning is useful
Deep learning is a part of machine learning
Artificial intelligence is changing the world
Natural language processing uses machine learning
Deep learning and artificial intelligence are related
I study natural language processing
Machine learning helps artificial intelligence"""

with open("corpus.txt", "w") as f:
    f.write(corpus_text)
```

```
from gensim.models import Word2Vec

with open("corpus.txt") as f:
    sentences = [line.lower().split() for line in f if line.strip()]

cbow = Word2Vec(sentences, vector_size=50, window=3, min_count=1,
                sg=0, epochs=200, seed=42, workers=1)   # sg=0 -> CBOW

word = "learning"
print(f"Vector for '{word}':\n", cbow.wv[word])
print("\nTop 3 words similar to", f"'{word}':")
for w, s in cbow.wv.most_similar(word, topn=3):
    print(f"  {w:<15}{s:.4f}")
```

```
Vector for 'learning':
 [-0.01265218  0.01312787  0.00568699  0.00067546 -0.00039841  0.01997441
 -0.02565439  0.01117282 -0.0126041  -0.02041918  0.00884585  0.02831058
  0.00549747  0.01508867  0.01037336  0.02101269  0.00334411 -0.02237608
  0.0118279  -0.0068095  -0.00343869 -0.01332745 -0.01474461  0.01838676
  0.02454898  0.00096197 -0.00397776  0.00756955 -0.00210602 -0.00235707
 -0.00026062 -0.00675901 -0.02064811  0.0071463   0.01516477 -0.01392712
  0.01324037  0.00752126 -0.01507423  0.01093039 -0.00584519  0.01468296
  0.01656358 -0.00365863 -0.01025395  0.01499949  0.00084411  0.01843737
  0.00935418  0.00387894]

Top 3 words similar to 'learning':
  intelligence   0.3630
  changing       0.2811
  uses           0.2573
```

```
skip = Word2Vec(sentences, vector_size=50, window=3, min_count=1,
                sg=1, epochs=200, seed=42, workers=1)   # sg=1 -> Skip-Gram

print("Vector for 'language':\n", skip.wv["language"])
```

```
print("\nTop 5 words similar to 'language':")
for w, s in skip.wv.most_similar("language", topn=5):
    print(f"  {w:<15}{s:.4f}")
```

```
Vector for 'language':
 [-3.16276168e-03  1.77079644e-02 -4.04974067e-04  1.06867636e-02
 -6.24552369e-04 -4.58689407e-03  4.37890273e-03  1.98613573e-02
 -1.05280653e-02  8.18031188e-03 -4.27469378e-03  1.53346853e-02
  8.51082988e-03  1.84297597e-03  1.09799253e-02 -2.44846125e-03
 -1.45936627e-02 -2.14204080e-02 -3.50073073e-03  1.22396005e-02
 -1.69535205e-02 -7.32351188e-03  6.85788784e-03 -1.02980714e-02
  1.76095851e-02 -1.08179366e-02  1.22362040e-02  8.45258590e-04
 -1.24796352e-03 -1.26912156e-02 -6.04483706e-04  1.67966988e-02
 -2.17247754e-02  1.30125312e-02 -9.80514451e-05  1.09789493e-02
  5.49309468e-03 -5.84617909e-03 -1.28760329e-02  8.02442990e-03
 -9.62015800e-03  5.90846129e-03 -1.02326619e-02  6.80811470e-03
  9.46135260e-03 -1.88428778e-02  1.24873593e-02 -1.91045832e-03
  1.25400778e-02 -2.30394788e-02]

Top 5 words similar to 'language':
  intelligence   0.3297
  uses           0.3219
  of             0.2813
  are            0.2801
  is             0.2763
```

```
model = Word2Vec(sentences, vector_size=50, window=3, min_count=1,
                 sg=1, epochs=200, seed=42, workers=1)

pairs = [("machine", "learning"), ("natural", "language"), ("deep", "learning")]
for a, b in pairs:
    print(f"Cosine similarity ({a}, {b}) = {model.wv.similarity(a, b):.4f}")
```

```
Cosine similarity (machine, learning) = 0.3256
Cosine similarity (natural, language) = 0.1505
Cosine similarity (deep, learning) = 0.2539
```